Angular

Ramana Reddy

Angular

- Angular is typescript based framework to design client side Single Page Applications (SPA)
- TypeScript is a superset of ECMAScript 6 (ES6)
- The main building blocks for Angular are modules, components, templates, metadata, data binding, directives, services and dependency injection
- Angular 2 is complete rewrite of Angular JS

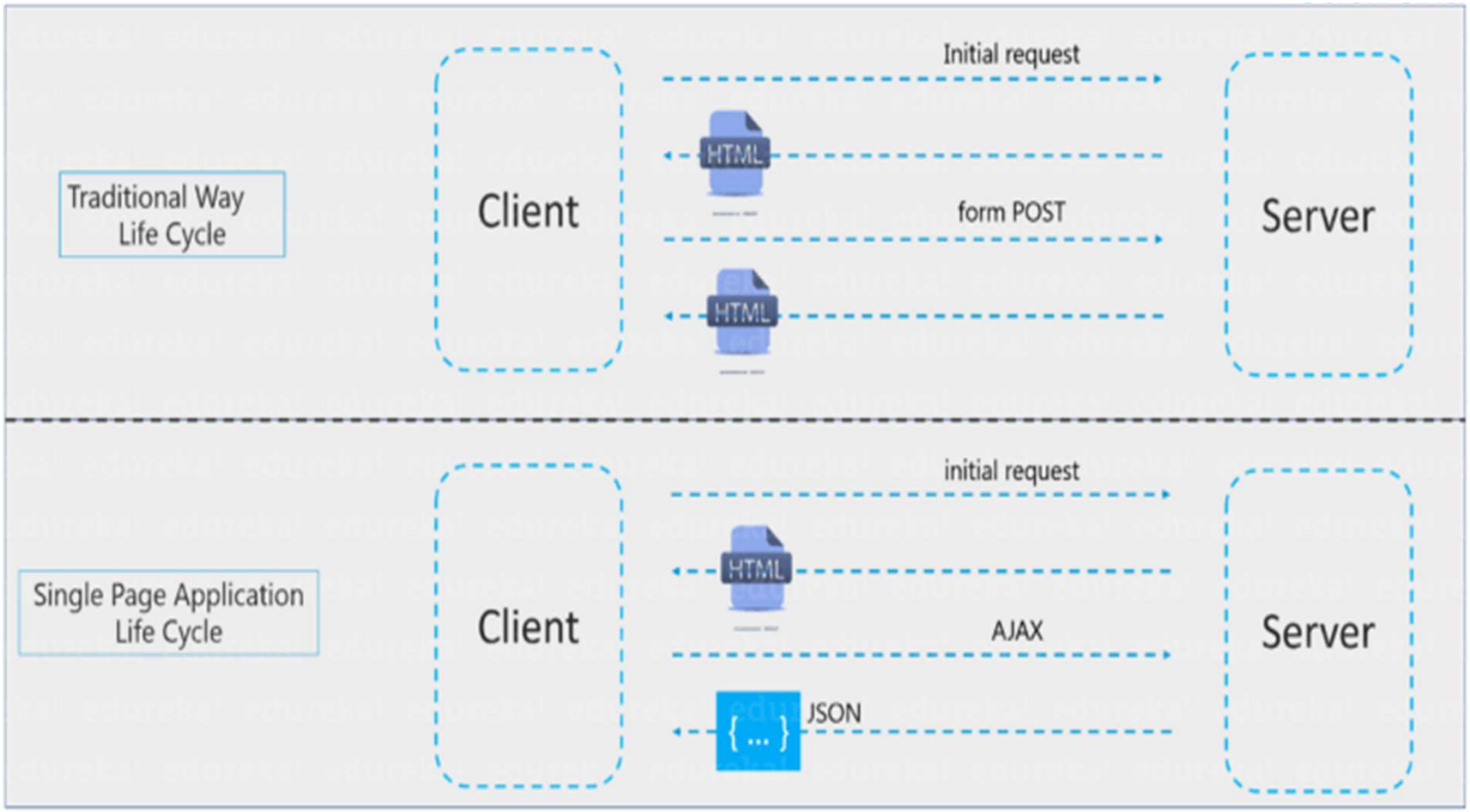
Angular Building Blocks

- The main building blocks of Angular are:
  - Modules
  - Components
  - Templates
  - Data binding
  - Directives
  - Services
  - Dependency injection

Single page Applications (SPA)

- Applications that are accessed via a web browser like other websites but offer more dynamic interactions
- The difference between a regular website and SPA is the reduced amount of page refreshes
- SPAs have a way to communicate with back-end servers without doing a full page refresh to get data loaded into our application
- As a result, the process of rendering pages happens mostly on the client-side

Single Page Applications (SPA)



Versioning and Release Schedule

- Angular version numbers indicate the level of changes that are introduced by the release.
- This use of semantic versioning helps us understand the potential impact of updating to a new version.

- Angular version numbers have three parts: major.minor.patch.
- For example, version 7.2.11 indicates major version 7, minor version 2, and patch level 11.

Versioning and Release Schedule

LEVEL OF CHANGE	DETAILS	SCHEDULE
Major release	Contains significant new features, some but minimal developer assistance is expected during the update. When updating to a new major release, you might need to run update scripts, refactor code, run additional tests, and learn new APIs.	Every six months
Minor release	Contains new smaller features. Minor releases are fully backward-compatible; but you can optionally modify your applications and libraries to begin using new APIs, features, and capabilities that were added in the release.	1-3 minor releases for each major release
Patch release	Low risk, bug fix release. No developer assistance is expected during update.	Almost every week

ng update

- ng update used to update the application to later versions.
- Check for instructions at update.angular.io
- It is advised to update one version at a time

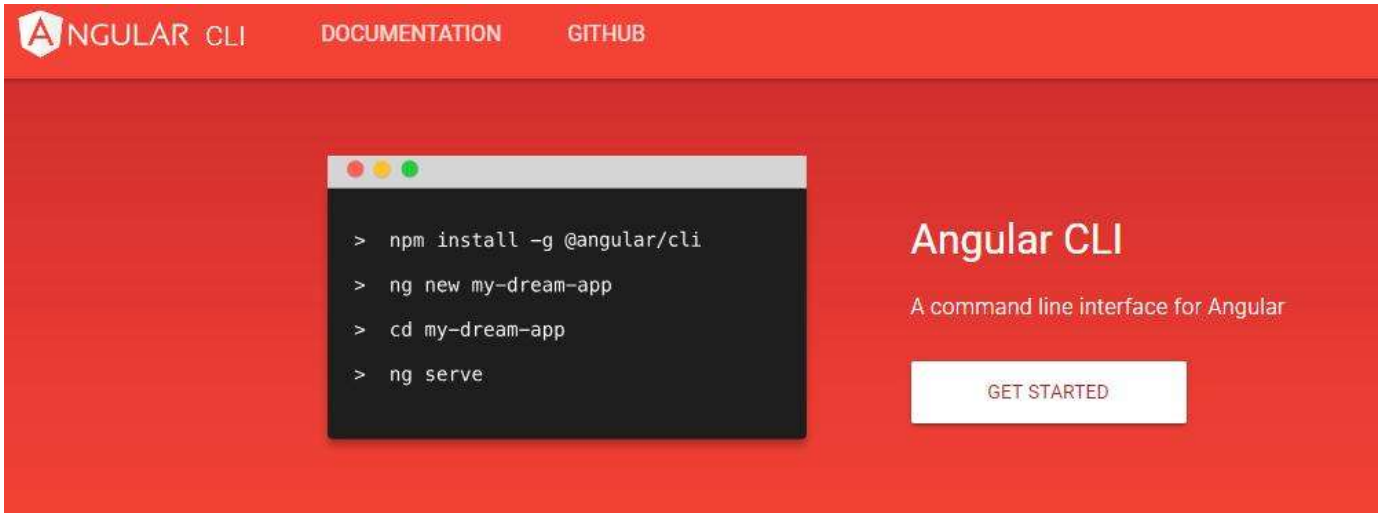
ng add

- ng add will use package manager to download new dependencies and invoke an installation script (implemented as a schematic) which can update our Angular project with configuration changes and adds additional dependencies like polyfills, or scaffold package-specific initialization code

- Example
  - ng add @angular/material

Angular Environment setup

- To install Angular, we require the following −
  - Nodejs
  - NPM
  - Angular CLI
  - IDE for writing your code (atom, Microsoft VS Code etc)
- Commands help available at cli.angular.io



Angular Project setup

- To install Angular

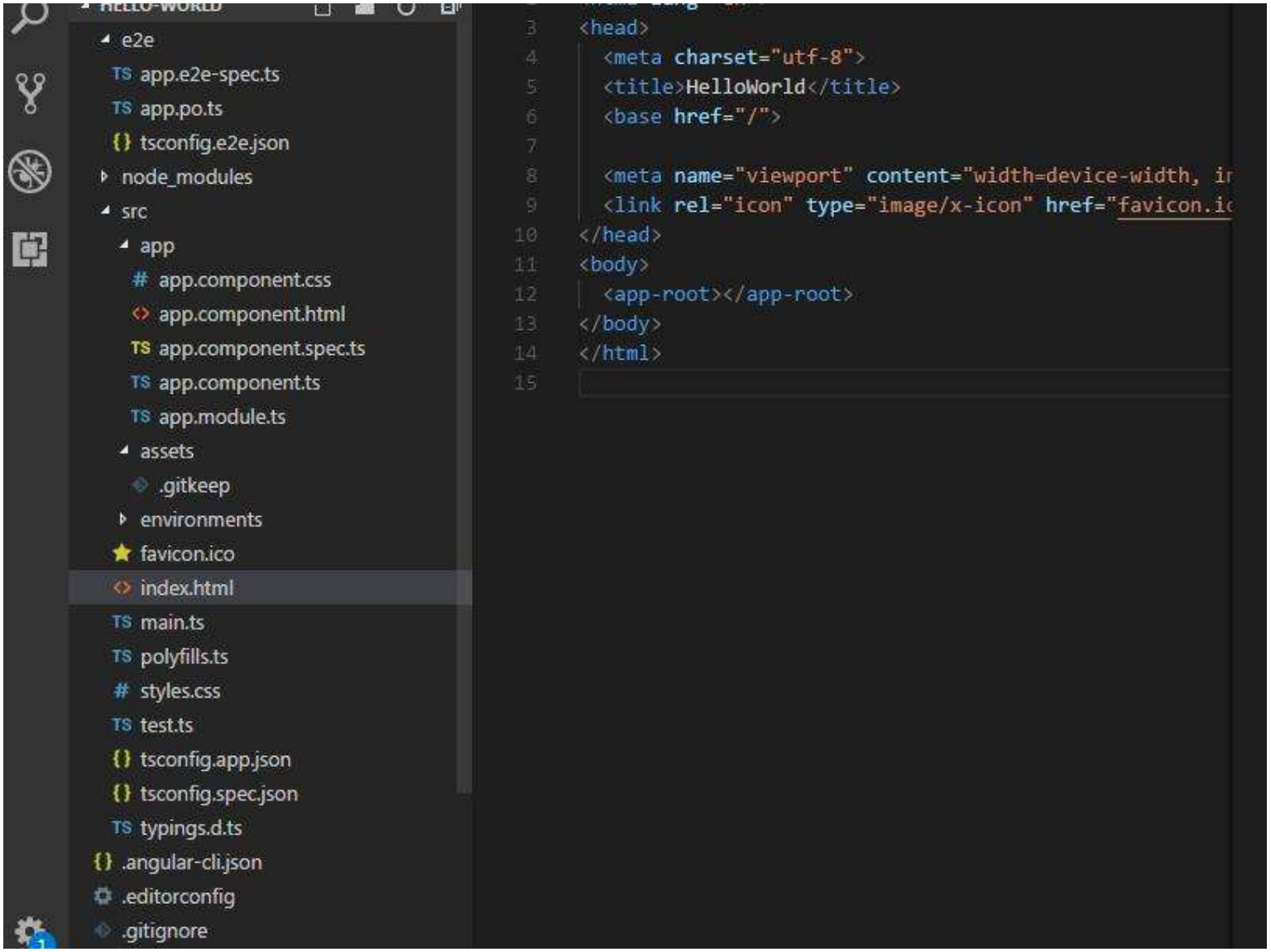
  npm install –g @angular/cli
- To start a workspace and new project

  ng new angular-test-project
- Creates a folder angular-test-project and creates a starter project
- Make the folder as current (Use CD command)

  cd angular-test-project
- Run the sample project

  ng serve
- Starts a server at port 4200

Angular Project Structure



Angular Project Structure

- **e2e** − end to end test folder. Mainly e2e is used for integration testing
- **node_modules** − all third party libraries required
- **src** − This folder is where we will work on the project using Angular
- **.angular.json** − It basically holds the project name, version of cli, etc.
- **.editorconfig** − This is the config file for the editor.
- **karma.conf.js** − used for unit testing via the protractor. All the information required for the project is provided in karma.conf.js file.
- **package.json** − The package.json file tells which libraries will be installed into node_modules when you run npm install.
- **protractor.conf.json** − tool for running end-to-end tests for Angular projects
- **tsconfig.json** − includes setting for the TypeScript compiler.
- **tslint.json** − includes the settings for TSLint which is a popular tool for linting TypeScript code. That means it checks the quality of our TypeScript code based on a few configurable parameters

Angular Project Structure

- **assets** -- folder to store assets like image files etc
- **evironment** -- Details of environment - production and development
- **index.html** - main file
- **main.ts** -- starting point - used in bootstrapping app module
- **polyfills.ts** – used to check browser compatibility
- **styles.css** -- Used to store global style sheet
- **test.ts** - - defines testing environment

Multiple Project in same workspace

- Create workspace without projects

  ng new app1 –createApplication=false directory=angularprojects
- *This does not create application (app1 is dummy). Simply create basic workspace with node-modules folder*
- *Move to workspace*

  cd angularprojects

- To create new applications

  ng g application app1
- This creates projects folder and application app1 inside it
- tslint and tsconfig json files extend those in workspace
- Create other applications in the same way
- angular.json is updated for each application. It also contains default project name
- To run default project

  ng serve
- To run a particular project app2

  ng serve -–project=app2

Angular Modules

- NgModules are containers for a cohesive block of code dedicated to an application domain, a workflow, or a closely related set of capabilities
- They can contain components, service providers, and other code files whose scope is defined by the containing NgModule.
- They can import functionality that is exported from other NgModules, and export selected functionality for use by other NgModule

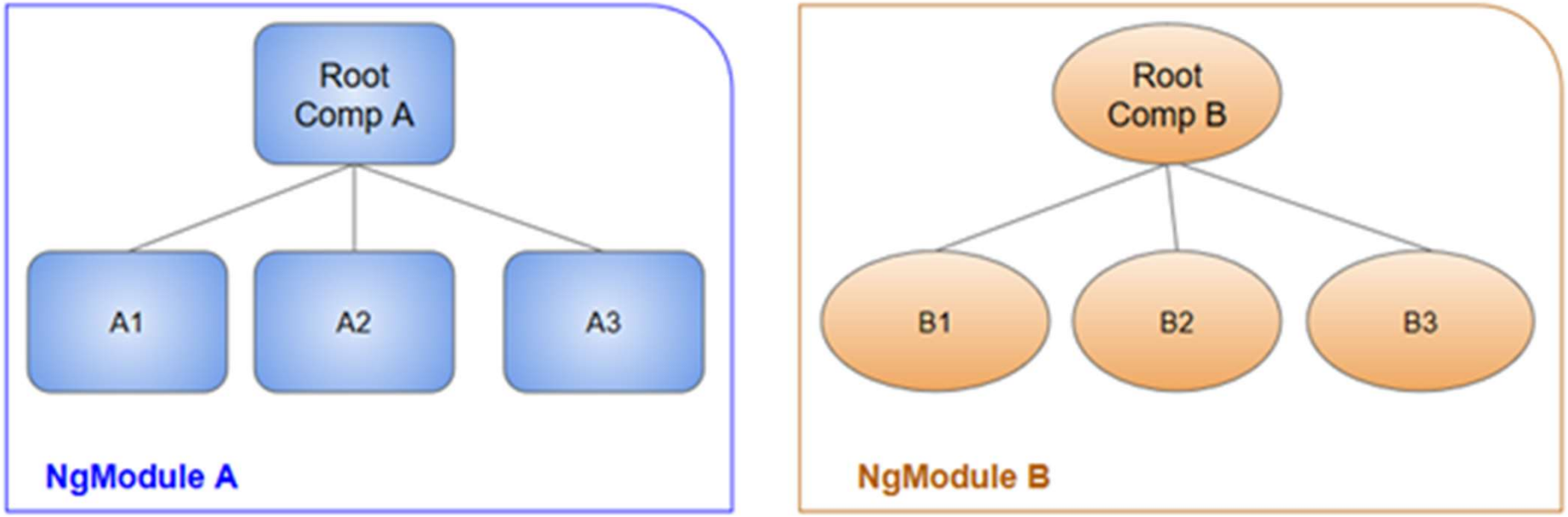
Angular Modules

```
@NgModule(
{
   imports: [ BrowserModule ],
   providers: [ Logger ],
   declarations: [ AppComponent ],
   exports: [ AppComponent ],
   bootstrap: [ AppComponent ] }
)
export class AppModule { }
```

Angular Modules

- ***Decorators*** are functions that modify JavaScript classes. used for attaching configuration metadata to classes
- Properties of NgModule decorator:
- ***declarations:*** classes that are related to views and it belong to this module
- There are three classes that can contain view: components, directives and pipes
- ***exports:*** The classes that should be accessible to the components of other modules
- ***imports:*** Modules whose classes are needed by the component of this module.
- ***providers:*** Services present in one of the modules which is to be used in the other modules or components
- ***bootstrap:*** The *root component* which is the main view of the application

Angular Modules



Feature Modules

- Feature modules are NgModules for the purpose of organizing code.
- A feature module collaborates with the root module and with other modules through the services it provides and the components, directives, and pipes that it shares

- Creating feature modules:

  ng generate module CustomerDashboard

- This will create a CustomerDashboardModule in customer-dashboard folder
- This will have CommonModule in imports which provides common directives like ngIf ngFor etc

Components in Feature Modules

- Creating components:

  ng generate component customer-dashboard/customer
- This will create a CustomerComponent in customer-dashboard folder and adds it to declarations of feature module

- This will have CommonModule in imports which provides common directives like *ngIf, *ngFor etc
- These components are not automatically available to other modules
- They have to be added to exports of the module

Shared Module

- Creating shared modules allows you to organize and streamline your code.
- You can put commonly used directives, pipes, and components into one module and then import just that module wherever you need it in other parts of your app

Building application

- Build modes
  - Development
  - Production
- Compiler modes
  - AOT (ahead of Time)
    - Compiles once
    - Happens with ng build --prod
  - JIT (Just in time)
    - Default for dev mode
    - Happens with ng build or ng serve

Angular Component

- The most fundamental building block in an Angular application is a *component*
- A component consists of three pieces:

- **HTML markup**: to render that view
- **State**: the data to display on the view
- **Behavior**: the logic behind that view. For example, what should happen when the user clicks a button.

Angular Component

- A component can have other components
- Each component can be maintained separately
- Naming pattern of a component :
- For a component named HelloComponent, the following files are associated :
  - hello.component.ts : component class code
  - hello.component.html : component template
  - hello.component.css : component's CSS styles
  - hello.component.spec.ts : includes unit tests
  - hello.module.ts : module details

- Angular creates AppComponent to start with and used as root component

App Component

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent {
  title = 'app';
}
```

Component decorator

app-root : where the component is placed

Components's html content and style sheet

class code

contains data and methods declaration

main.ts

- main.ts is the file from where we start our project development. It starts with importing the basic module which we need

```
import { enableProdMode } from '@angular/core';
import { platformBrowserDynamic } from '@angular/platform-browser-dynamic';

import { AppModule } from './app/app.module';
import { environment } from './environments/environment';

if (environment.production) {
  enableProdMode();
}

platformBrowserDynamic().bootstrapModule(AppModule)
  .catch(err => console.log(err));
```

Creating new component

- To create a new component :

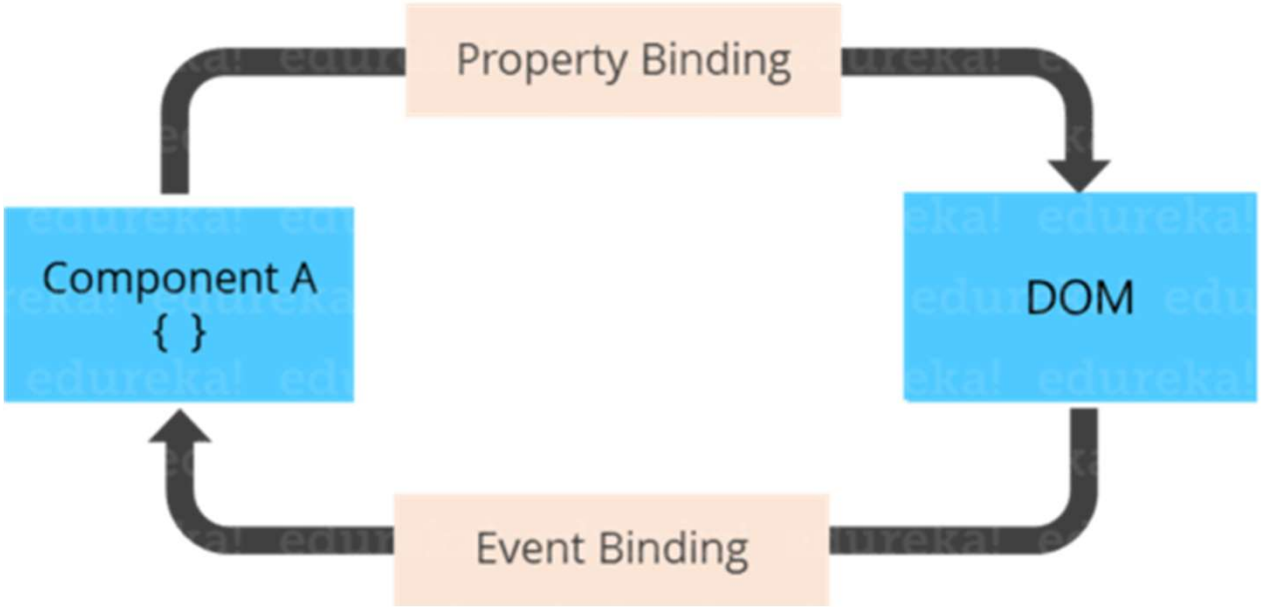
```
ng  generate  component  name
        or
ng g c name
```

- Creates all files for the component and also updates module to include the component

Binding

Data Binding

- **data binding** is a mechanism for coordinating parts of a template with parts of a component
- You should add binding markup to the template HTML to tell Angular how to connect both sides



Data Binding

- Data binding done with interpolation of data declared in component class

  {{data}}

- This is same as

  <p [innerHTML]='data' />

- we re binding innerHTML property to value of data using "[ ]".

- Binding of other properties of the component is possible in the same way

Event Binding

- For event binding use “( )” and bind to a method which is called on the event

  <button (click)=”calculate()” >Click me</button>

- calculate() function is called on click event

- Event object can be passed in the method

  <button (click)=”calculate($event)” >Click me</button>

- In the method of component $event should be the argument

Template Variable

- Any element can be referred with template variable
- Referred as method argument for events used in the code

```
<input #email (click)= 'changeValue(email.value)' />
```

```
<input #name />
<button (click)='display(name.value)'>Click</button>
```

- Never pass 'this' as argument. It refers the component

Data Binding – attribute or property?

- All html attributes are not dom element properties
- [ ] can be used only for properties

<img [src]="imgUrl" /> // works fine as src is property and attribute

<td [colspan] ='size' >… </td> // does not work as colspan is only attribute not the property

- Binding attributes is done in different way

<td [attr.colspan] ='size' >… </td> // works fine

Two way Data Binding

- Combines property and event binding in a single notation, using the **ngModel** directive

```
<input  [(ngModel)]="person.name">
```

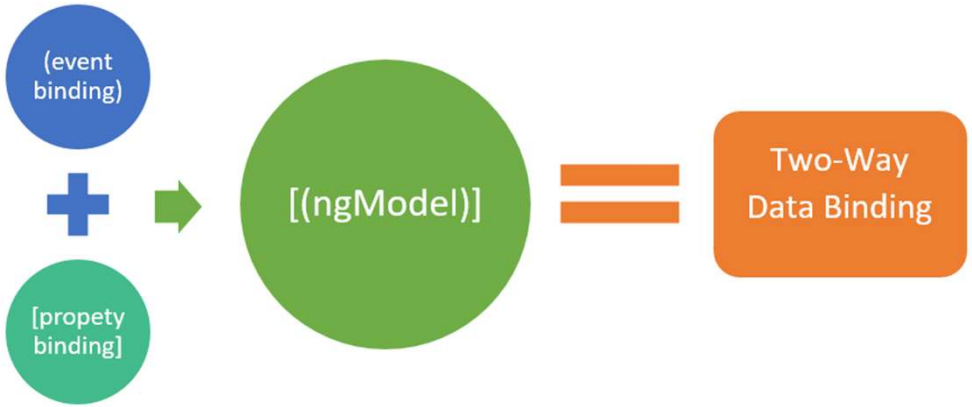
- In two-way binding, a data property value flows to the input box from the component as with property binding

- The user's changes also flow back to the component, resetting the property to the latest value, as with event binding

- Angular processes all data bindings once per JavaScript event cycle, from the root of the application component tree through all child components

Two way Data Binding

- [(ngModel)] which is also referred as ‘Banana in a Box’ is a combination of property binding and event binding




```
<input  [(ngModel)]=“name">
```

is same as:

```
<input  [value]="name"  (input)="name=$event.target.value" />
```

Two way Data Binding

- We need FormsModule for using ‘ngModel’
- Import FormsModule and also add it to imports in the module file
- app.module.ts file:

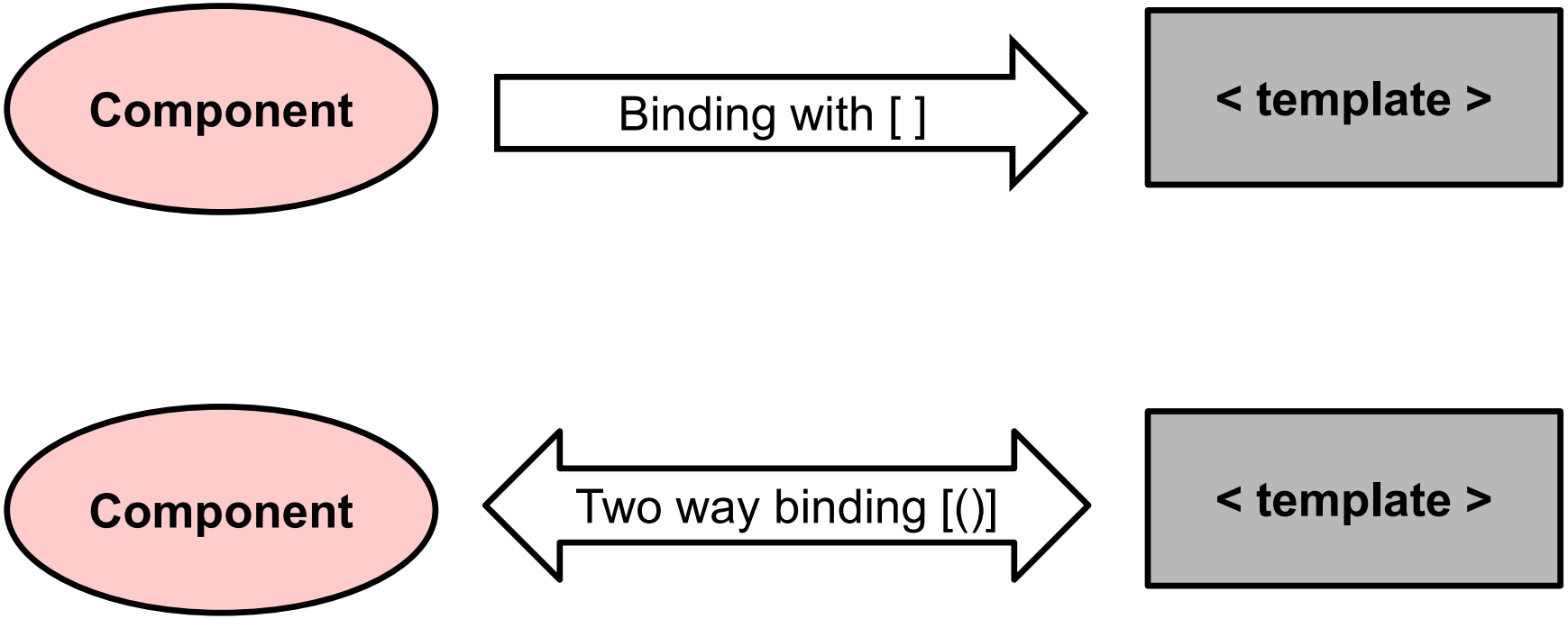
```
import { AppComponent } from './app.component';
import { CoursesComponent } from './courses/courses.component';
import { CoursesService } from './courses.service';

import { FormsModule } from '@angular/forms';


@NgModule({
  declarations: [
    AppComponent,
    CoursesComponent
  ],
  imports: [
    BrowserModule,
    FormsModule
  ],
```

Two way Data Binding

- Using radio group:

```
<label>
    <input type='radio' value='Male' [(ngModel)]='gender’/>
    Male
</label>
<label>
   <input type='radio' value='Female' [(ngModel)]='gender’/>
   Female
</label>
```

Binding


Data Flow
Component
Binding with [ ]
< template >
Component
Two way binding [()]
< template >

Directives

Directives

- Components are rendered to DOM elements according to the instructions given by **directives**

- A directive is a class with a **@Directive** decorator
- A component is a *directive-with-a-template*
- @Component decorator is actually a @Directive decorator extended with template-oriented features

- Two *other* kinds of directives exist:
  - ***structural*** directives
  - ***attribute*** directives

Structural Directives

- **Structural** directives alter layout by adding, removing, and replacing elements in DOM
- *ngFor : loops through collection to render for each value
- *ngIf : used to decide whether DOM element has to be rendered or which template is rendered

- ngSwitch : has switch case structure(this is attribute directive)
- *ngSwitchCase : refers one case in switch
- *ngSwitchDefault : refers default of switch

Structural Directives : Examples

- element displayed only if condition is true

```
<p *ngIf='isTrue'>Value is true</p>
<ul  *ngIf='array.length>4'>
<li  *ngFor="let x of array">{{x}}</li>
</ul>
```

- Using else:

```
<p *ngIf='isTrue; else templ'>First value</p>
<ng-template #templ>
  <h2>second value</h2>
</ng-template>
```

- element displayed if condition true else template displayed
- semicolon required with boolean value
- template name is defined with #

Structural Directives : Examples

- templates can be mapped for if and else. element is ignored

```
<p *ngIf='isTrue; then template1 else template2'>First value</p>
<ng-template #template1><h2>First value</h2></ng-template>
<ng-template #template2><h2>second value</h2></ng-template>
```

- ngFor

```
<ul>
<li *ngFor="let x of array">{{x}}</li>
</ul>

<ul>
<li *ngFor="let p of persons; let i=index">{{i + 1}} - {{p.name}}</li>
</ul>
```

Structural Directives : Examples

- ngSwitch

```
<div [ngSwitch]=“person.gender">
  <p  *ngSwitchCase="'M'">Male</p>
  <p  *ngSwitchCase="‘F'">Female</p>
  <p  *ngSwitchDefault>Unspecified</p>
</div>
```

What is ‘*’ in these directives?

- asterisk is "syntactic sugar" for something a bit more complicated internally
- *ngIf *attribute is translated* into a <ng-template> *element*, wrapped around the host element , like this

```
<div *ngIf="person.age>25">
   {{person.name}}
</div>
```

- Converted to:

```
<ng-template  [ngIf]  ="person.age>25">
  <div >{{person.name}}</div>
</ng-template>
```

Attribute Directives

- **Attribute** directives alter the appearance or behavior of an existing element
- In templates, they look like regular HTML attributes

```
<input  [(ngModel)]="movie.name">
<button  (click)=”calculate()” >Click me</button>
```

What else about directives

- Two structural directives cannot be used in the same element
- Below one is not allowed

```
<div *ngFor='let p of persons' *ngIf='value>10'>….. </div>
```

- Attribute directive can be combined with structural directive

```
<ul>
    <li *ngFor='let p persons'  [NgSwitch]='p.gender'>
        <p *ngSwitchCase='M'>Male</p>
        <p *ngSwitchCase='F'>Male</p>
        <p *ngSwitchDefault>Unknown</p>
    </li>
</ul>
```

Create your own Directives

- Custom directives are user defined directives and are not standard
- Created with the command

```
ng  g  d  <name>
```

- This command creates the directive and also registers the directive with the module
- Example:

```
ng  g  d textChange
```

- output:

```
create src\app\text-change.directive.spec.ts
create src\app\text-change.directive.ts
update src\app\app.module.ts
```

Directive entry in the module

```
import { TextChangeDirective } from './text-change.directive';

@NgModule({
  declarations: [
    AppComponent,
    NewCmpComponent,
    TextChangeDirective
  ],

   ……................
})

export class AppModule { }
```

TextChangeDirective

- Contains class and @Directive decorator
- Whatever we define in the selector, the same has to match in the view, where we assign the custom directive

```
import { Directive } from '@angular/core';

@Directive({
  selector: '[textChange]'
})

export class TextChangeDirective {
  constructor() { }
}
```

```
<div style="text-align:center">
  <span textChange >Welcome to Angular</span>
</div>
```

TextChangeDirective

- Constructor of the class will have argument of type ElementRef which gives details of the element
- Properties of the element can be controlled with Elementref

```
import {ElementRef} from '@angular/core‘

export class TextChangeDirective {

  constructor(private element: ElementRef) {
        element.nativeElement.innerText=“new text";
  }

}
```

@HostListener

- The @HostListener decorator lets you subscribe to events of the DOM element that hosts an attribute directive
- With @HostListener, we can control the behavior of the element based on events
- Prefix @HostListener with an event to the method which should be executed on that event on the element

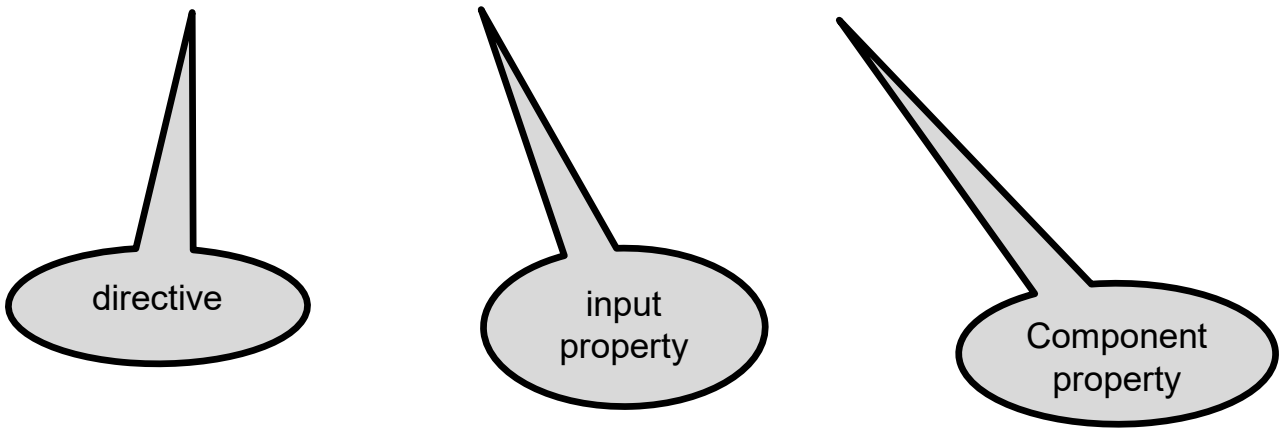
```
constructor(private el:ElementRef){}

@HostListener('mouseenter') onMouseEnter() {
    this.el.nativeElement.style.backgroundColor = color;
}

@HostListener('mouseleave') onMouseLeave() {
    this.el.nativeElement.style.backgroundColor = color;
}
```

Passing values to Directive through @Input

- To pass values to directive create properties with @input () and pass the value from the template

```
@Input("color")  InputFormat:string;
```

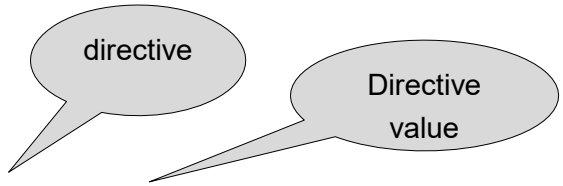
```
<Input  highlight  [color]="selectedColor" />
```



Passing values - Example

```
@Directive({
        selector: '[highlight]'
})

export class HighlightDirective {

constructor(private el: ElementRef) { }
@Input() default: string;
@Input(‘color') highlightColor: string;

@HostListener('mouseenter') onMouseEnter() {
    this.highlight(this.highlightColor || this.default || 'red');
}

@HostListener('mouseleave') onMouseLeave() {
        this.highlight(null);
}

private highlight(color: string) {
    this.el.nativeElement.style.backgroundColor = color;
}
```

```
<div>
  <input type="radio"  (click)=“selectedColor='lightgreen'">Green
  <input type="radio"  (click)=“selectedColor='yellow'">Yellow
  <input type="radio"  (click)=“selectedColor='cyan'">Cyan
</div>
```




```
<p highlight [color]=‘selectedColor’>
    This line highlighted with selected color
<.p>

<p highlight [default]=“’brown’”>
This line highlighted with default color
<.p>

Highlight me too!
```

Single value in a Directive

- If the directive has single value , we can bind it with the same name and no separate property needed

```
@Input("color")  InputFormat:string;
```

```
<Input  [highlight]='selectedColor' />
```

Pipes

Pipes

- Pipes are filters to transform the data
- The | character is used to transform data
- Example:

```
<p>{{title | uppercase}} </p>
```

- Angular provides many built-in pipes. Some are listed below −
  - lowercase
  - uppercase
  - date
  - currency
  - json
  - number
  - Slice
  - ……

Pipes with parameters

- A pipe can accept any number of optional parameters to fine-tune its output
- To add parameters to a pipe, follow the pipe name with a colon (:) and then the parameter value (such as currency:'EUR')
- If the pipe accepts multiple parameters, separate the values with colons (such as slice:1:5)

- Examples:

```
<p>Birthday : {{ birthday | date:"MM/dd/yy" }} </p>

<p>Birthday : {{ birthday | date:shortDate}}</p>

<ul>
    <li *ngFor="let name of  names | slice :1:3">{{name}}</li>
</ul>
```

Pipes parameters – number and currency

- Decimal representation options, specified by a string in the following format:
  {minIntegerDigits}.{minFractionDigits}-{maxFractionDigits}

- Currency can specify symbol
- Examples:

```
<p>Salary is {{ salary | number:6.0-2 }} </p>
<p>Number :  {{amount|number:'7.0-3'}}</p>
<p> currency : {{amount|currency}}</p>     // default symbol
<p> Currency:# {{amount|currency:'#'}}</p>  with  # as symbol
<p>currecny: {{amount|currency:'#':'4.0-3'}}</p>
```

Pipes parameters – number and currency

- You can chain pipes together in potentially useful combinations
- In the following example, to display the birthday in uppercase, the birthday is chained to the DatePipe and on to the UpperCasePipe
- The birthday displays as APR 15, 1988

The chained birthday is {{ birthday | date | uppercase}}

Custom Pipes

- You can write your own custom pipes
- Create with ng g p pipename and register with module
- Here's a custom pipe named ExponentialPipe that can raise a number to given power

```
import { Pipe, PipeTransform } from '@angular/core';

@Pipe({name: 'exponential'})
export class ExponentialPipe implements PipeTransform {

  transform(value: number, exponent: string): number {
    let exp = parseFloat(exponent);
    return Math.pow(value, isNaN(exp) ? 1 : exp);
  }
```

Custom Pipes – How?

- A pipe is a class decorated with pipe metadata.
- Class has to implement PipeTransform and provide transform method
- transform method that accepts an input value followed by optional parameters and returns the transformed value

```
import { Pipe, PipeTransform } from '@angular/core';

@Pipe({name: 'exponential'})
export class ExponentialPipe implements PipeTransform {

  transform(value: number, exponent: string): number {
    let exp = parseFloat(exponent);
    return Math.pow(value, isNaN(exp) ? 1 : exp);
  }
```

Use:
{{value|exponential:'3'}}